Kantonsschule Im Lee Winterthur

Maturitätsarbeit HS 2026/27

Zugängliche Informationen trotz Verschlüsselung, die Macht der Metadaten.

Bernard Rognon, Tim Frauenfelder 4f

Betreut durch: Thomas Graf

Winterthur

19. Oktober 2026

Inhaltsverzeichnis

1 Einleitung **2**

2 Motivation **3**

3 Die Client-Server-Struktur **4**
3.1 Das Funktionsprinzip der Client-Server-Struktur 4
3.1.1 Der Aufbau 5
3.1.2 Verbindung zwischen Client und Server 5
3.1.3 Kryptologie 5
3.2 Vergleich zu anderen Strukturen 6
3.3 Die umsetzung von Signal/WhatsApp 6

4 Die Metadaten **7**

5 Unser Programm **8**
5.1 Architektur und Systemübersicht 8
5.1.1 Das Client-Server-Modell 8
5.1.2 Kommunikationsprotokoll 8
5.1.3 Datenfluss 8
5.2 Server-Hardware und Betriebsumgebung 9
5.2.1 Hardware-Basis 9
5.2.2 Ressourcenbelastung und Netzwerkanbindung 9
5.2.3 Betriebssystem und Programmiersprache 9
5.3 Server-Side: Das Backend 9
5.3.1 Netzwerk und Verbindungsmanagement 9
5.3.2 Datenpersistenz und Datenbank 9
5.3.3 Sicherheit und Authentifizierung auf Serverseite 9
5.4 Clientside: Benutzeroberfläche und Client-Logik 9
5.4.1 Verbindungsaufbau 9
5.4.2 Asynchrone Nachrichtenverarbeitung 9
5.4.3 Kryptographie und Datenintegrität 9
5.5 Zusammenfassung 9

6 Metadaten in unserem Programm **10**

7 Reflexion **11**

8 Future Work **12**

9 Fazit **13**

Abbildungsverzeichnis **14**

Quellenverzeichnis **15**

Kapitel 1

Einleitung

Kapitel 2

Motivation

Im September 2025 sah man plötzlich im ganzen Internet Nachrichten zur Child Sexual Abuse Regulation (CSAR) der EU oder einfach EU Chat Control 2.0. In diesem Gesetzentwurf geht es darum, dass in den EU Staaten jede einzelne versendete Private nachricht und Datei von einem programm geprüft wird auf gefährliche Inhalte. Selbstverstädlich machte dies vielen Leuten angst um ihre Privatsphäre da es Ende-zu-Ende-Verschlüsselungen umging und eine permanente Überwachung aller EU Bürger erlaubte. Die andere Sorge war das unabhängige Staaten danach auch eigene Gesetze zur überwachung von online Kommunikation erlassen würden. Tim und ich haben auch über dieses Thema disskutiert und kamen zum schluss das wir zusammen eine eigene Platform zum chatten programmieren wollen auch wenn wir nicht direkt betroffen währen von der CSAR. Am nächsten Morgen haben wir ganz Tutti und Ricardo durckkämmt auf der suche nach einem alten PC welchen wir zu einem Linux (Arch btw) Homeserver umbauen können. Eine Woche später und 50 CHF ärmer waren wir in Rüti ZH um einen Lenovo Office-PC abzuholen.

Anfangs Januar hatten wir noch keine Zeile Code geschreiben. Wir hatten jedoch einen Plan wie wir zwei Fliegen mit einer Klappe schlagen konnten, denn Tim und ich hatten beide noch keine Ahnung zu welchen Thema wir unsere Maturitätsarbeit wollten. Unserem Plan folgend fragten Thomas Graf ob es möglich ist die Maturitätsarbeit zu zweit zu schreiben. Als es ja sagte war es für uns beide klar: dieses Projekt wird zu unserer Maturitätsarbeit. Zusammen schreiben wir in Python ein möglichst sicheres Chat Programm welches wir dann auf unserem Server hosten können. Somit haben wir sicher die motivation das Projekt abzuschliessen oder zumindest den Druck. Das einzige das uns noch fehlte war einen Schwerpunkt für die Arbeit. Auf einer langen Zugfahrt nach Interlaken sammelten wir viele Ideen die wir sofort wider verworfen, der grund dafür war meistens das sie zu viel Mathematik involvierten. Im endeffekt kamen wir zum schluss das wir uns damit beschäftigen wollen wie man die Menge an sammelbaren Metadaten minimalisieren oder sogar komplett eliminieren kann. Da dies ein sehr aktuelles Thema ist und Metadaten zu sammeln die einfachste art ist private Informationen aus chatplatformen zu gewinnen.

Kapitel 3

Die Client-Server-Struktur

Als erstes, wollen wir verstehen, wie ein Messaging-Programm grundsätzlich funktioniert. Unser Ziel ist es, von einem Gerät A eine Nachricht an ein anderes Gerät B versenden zu können. Dies sollte möglichst wenig Leistung gebraucht werden, möglichst schnell stattfinden, und die Nachricht darf für Niemanden ausser dem Absender und den Empfänger lesbar sein. Um dies zu tun, gibt es schon sehr viele Programme. Die meisten können jedoch anhand ihres Funktionsprinzips in drei Unterkategorien eingeteilt werden. Diese sind die Client-Server-Struktur, das Peer-to-Peer-Messaging und das föderierte Servermodell.

3.1 Das Funktionsprinzip der Client-Server-Struktur

Die Client-Server-Struktur ist das verbreitetste Modell für Messaging-Programme und auch dasjenige, welches wir für unser Projekt benutzen. In diesem Modell werden Clients benutzt, die sich alle mit einem Server verbinden[1]. Der Client ist ein Programm, das auf dem Gerät jedes Nutzers läuft. Der Client dient dazu, Nachrichten an den Server zu senden und sie von ihm zu empfangen. Dazu hat er ein Interface, auf dem die Nachrichten angezeigt werden und neue verfasst werden können. Der Server ist eine zentrale Rechenmaschine, welche gleichzeitig mit allen Clients verbunden ist. Er speichert die vom Client verschickten Nachrichten ab, bis er eine Verbindung zum gewählten Empfänger erstellt hat und leitet sie dann an ihn weiter. Zusätzlich kann dieser, wenn er die Nachricht empfängt und liest, dies dem Server mitteilen, und dieser leitet dies an den Sender zurück. Bekannte Beispiele für Programme, die diese Struktur verwenden, sind WhatsApp und Signal. Einige Programme, wie zum Beispiel Discord, löschen die Nachrichten nicht vom Server, nachdem sie weitergeleitet wurden. Dies ermöglicht dem Nutzer, alle vergangenen Nachrichten auf einem neuen Gerät abzurufen.

[1] Dies kann auch einen Server Cluster sein

3.1.1 Der Aufbau

3.1.2 Verbindung zwischen Client und Server

3.1.3 Kryptologie

Wenn ein Messaging-Programm Nachrichten direkt verschickt, erlaubt das allen, sie auch zu lesen, solange man Zugriff auf das Netzwerk der beteiligten Clients oder des Servers hat. Dies ermöglicht keine Privatsphäre. Um das zu verhindern, benutzt man ein Verfahren namens Verschlüsselung.

Die Definition von Wikipedia:

> *"Verschlüsselung nennt man die Umwandlung von Informationen, genannt Klartext, in einen Geheimtext. Dies geschieht mithilfe von kryptografischen Verfahren unter Verwendung eines geheimzuhaltenden Schlüssels, der nur den befugten Personen bekannt sein darf. Mit Kenntnis des Schlüssels kann der Geheimtext wieder in den Klartext zurückgewandelt werden, was Entschlüsselung genannt wird."*

Um in einem Messaging-Programm Privatsphäre garantieren zu können, benötigt man eine Ende-zu-Ende-Verschlüsselung. Das bedeutet, dass die Nachricht am Ende des Absenders verschlüsselt wird und nur am anderen Ende, dem des Empfängers, entschlüsselt werden kann. Niemand dazwischen darf die Nachricht entschlüsseln können. Der Server erhält also nur eine verschlüsselte Version Nachricht. Um die verschlüsselte Nachricht wieder zu entschlüsseln, braucht der Empfänger jedoch zwingend den Schlüssel. Dies ist jedoch bei einer symmetrischen Verschlüsselung, eine Verschlüsselung bei der der gleiche Schlüssel zur Entschlüsselung wie zur Verschlüsselung verwendet wird, ein Problem. Da der Empfänger den Schlüssel haben muss, um die Nachricht zu entschlüsseln, müsste der Schlüssel auch mitgeteilt werden. Dabei hätte aber jeder Zugriff auf den Schlüssel und die verschlüsselte Nachricht und könnte sie somit selbst entschlüsseln. Um dieses Problem zu, lösen benutzt man eine asymmetrische Verschlüsselung.

Die asymmetrische Verschlüsselung

Die asymmetrische Verschlüsselung zeichnet sich dadurch aus, dass sie nicht den gleichen Schlüssel zur Verschlüsselung und Entschlüsselung verwendet. Um dies zu erreichen, gibt es verschiedene Algorithmen, die alle dem gleichen Prinzip folgen. Bei der Erstellung eines Accounts generiert der Client A ein sogenanntes Key-Pair. Dieses besteht aus einem Public und einem Private Key. Der Public Key wird offen an alle anderen Nutzer versendet, der Private Key wird lediglich auf dem Gerät des Nutzers A gespeichert. Wenn Client B eine Nachricht an Client A versenden möchte, benutzt er den Public Key von Client A, um diese zu verschlüsseln. Dieser kann aber nicht verwendet werden, um die Nachricht wieder zu entschlüsseln. Nur mit dem Private Key von Client A ist es möglich, die Nachricht wieder zu entschlüsseln. So wird eine sichere Kommunikation ermöglicht, ohne die Schwachstelle, dass ein Schlüssel zur Entschlüsselung ausgetauscht werden muss. Solange alle Private Keys geheim bleiben, ist die Privatsphäre garantiert. Der Nachteil an diesem Verfahren ist, dass die Algorithmen für die asmmyetrische Verschlüsselung viel langsamer sind und mehr Rechenleistung brauchen.

Das Hybrid-Kryptosystem

Als letzte art der verschlüsselung werden wir das hybrid kryptosystem anschauen. Sie ist wie der name schon Andeutet eine verbindung aus der asymmetrischen und symmetischen Verschlüsselung. wenn Client A eine nachticht versendet wird zuerst ein zufälleriger session-key genereirt. Dieser wird dann benutzt um die nachricht, die grosse datenmenge, effizient symetrisch zu verschlüsseln. Danach wird der session-key asymetrisch verschlüsselt mit dem Public-Key von Client B. da der session key sehr klein ist wird trots asymetrischer verschlüsselung wenig zeit und rechenleistung verbraucht. Danach werden die Verschlüsselte nachticht und der Verschlüsselte Session key an Client B versendet. Dieser entschlüsselt zuerst den Session kex mit seinem Provate key und kann dann die nachricht mit dem session key symetrisch entschlüsseln. Bei diesem vorgang wird die grosse datenmenge effizient verschlüsselt mit der symetrischen verschlüsselung und ist trotzdem sicher da der session key asymetrisch verschlüsselt wird.

3.2 Vergleich zu anderen Strukturen

P2P und blah blah sind au mega cool

3.3 Die umsetzung von Signal/WhatsApp

Kapitel 4

Die Metadaten

Kapitel 5

Unser Programm

5.1 Architektur und Systemübersicht

5.1.1 Das Client-Server-Modell

Für unseren Ansatz eines Messengers haben wir uns aus vielen rein praktischen Gründen für ein Client-Server Modell entschieden. Denn da die meisten modernen Messenger ebenfalls so funktionieren hatten wir somit Zugang zu vielen Informationen von bereits funktionierenden Systemen. Dazu kommt das wir bereits im besitz eines Servers (ein alter Büro-Pc) waren.

5.1.2 Kommunikationsprotokoll

Für die Kommunikation zwischen Server und Client sind TCP-Sockets zuständig. Diese TCP-Sockets erstellen eine Verbindung zwischen den beiden Geräten mithilfe von IP-Adressen und Ports. Wie es der Name schon sagt nutzen die TCP-Sockets das Kommunikationsprotokoll TCP (Transmission Control Protocol). Das TCP nutzt ein lokaler TCB (Transmission Control Block), in diesem sind wichtige Informationen gespeichert, unter anderem die IP-Adresse und den Port des Partner Sockets, den Verbindungsstatus (verbunden, wartend, geschlossen, etc.), welche Daten noch gesendet oder gelesen werden müssen und die Sequence Numbers welche die Reihenfolge der Empfangenen und Versendeten Bytes festhalten. Dieser letzte Punkt ist für unsere Anwendung der Punkt der dieses Kommunikationsprotokoll von anderen Abhebt, denn so kann man sicher sein das die zu versendenden Daten vollständig und Fehlerfrei ankommen in dem man einfach die Sequence Numbers vergleicht.

5.1.3 Datenfluss

Wenn jetzt jemand den Messenger öffnet und eine Nachricht versenden will wird erst eine TCP Verbindung zum Server aufgebaut und die Identität des Absenders überprüft. Dann wird die Nachricht noch lokal verschlüsselt und in die Nachrichtenstruktur eingebunden. Wenn jetzt jemand den Messenger öffnet und eine Nachricht versenden will wird erst eine TCP Verbindung zum Server aufgebaut und die Identität überprüft. Dann wird die Nachricht noch lokal verschlüsselt und in die Nachrichtenstruktur eingebunden. Im ersten Teil wird festgehalten welchem Zweck die Nachricht dient, 0 steht für Login oder Account erstellen, 100 steht für eine Statusnachricht an

den Server und 200 für eine Nachricht an einen anderen Benutzer. All diese Nachrichten Kategorien brauchen dann unterschiedliche weitere Informationen wie die Nachricht, die Zeit oder den Sessionkey welcher die Unterschrift für das gesendete darstellt. Erst jetzt wird dieses ganze Paket an den Server versendet, dort wird es entpackt und in die einzelnen Bestandteile aufgetrennt. Jetzt in dem Fall das es sich um eine Nachricht an einen anderen Benutzer handelt schaut der Server ob dieser Benutzer im Moment online ist und falls ja verpackt er die Nachricht wieder mit den notwendigen Informationen und sendet sie weiter. Falls der Empfänger nicht online sein sollte wird die Nachricht auf dem Server gespeichert, bis der Empfänger sich das nächste mal mit dem Server verbindet.

5.2 Server-Hardware und Betriebsumgebung

5.2.1 Hardware-Basis

Als Grundlage für unseren Server dient ein Lenovo Thinkcenter mit eînem Intel i7-xxxx und 8GB Arbeitsspeicher, auch wenn das nach nicht sonderlich viel klingt ist es trotzdem mehr als genug Rechenleistung um mehrere Benutzer gleichzeitig zu handhaben. Dazu kommt das ein solcher kleiner PC für die Last die wir erwarten viel effizienter ist.

5.2.2 Ressourcenbelastung und Netzwerkanbindung

5.2.3 Betriebssystem und Programmiersprache

5.3 Server-Side: Das Backend

5.3.1 Netzwerk und Verbindungsmanagement

5.3.2 Datenpersistenz und Datenbank

5.3.3 Sicherheit und Authentifizierung auf Serverseite

5.4 Clientside: Benutzeroberfläche und Client-Logik

5.4.1 Verbindungsaufbau

5.4.2 Asynchrone Nachrichtenverarbeitung

5.4.3 Kryptographie und Datenintegrität

5.5 Zusammenfassung

Kapitel 6

Metadaten in unserem Programm

Kapitel 7

Reflexion

Kapitel 8

Future Work

Kapitel 9

Fazit

Abbildungsverzeichnis

Quellenverzeichnis

[1] fhdrsdg. *How to expand a frame inside a canvas in tkinter*. 2018. URL: https://stackoverflow.com/questions/50243679/how-to-expand-a-frame-inside-a-canvas-in-tkinter (besucht am 10. 10. 2026).

[2] GeeksforGeeks. *Scrollable Frames in Tkinter*. 2026. URL: https://www.geeksforgeeks.org/python/scrollable-frames-in-tkinter/ (besucht am 10. 10. 2026).

[3] Peter Grill. *When to use \par and when \newline or blank lines?* URL: https://tex.stackexchange.com/questions/82664/when-to-use-par-and-when-newline-or-blank-lines (besucht am 13. 10. 2026).

[4] Harshith. *Deep Dive into TCP Sockets: How Data Travels Under the Hood*. 2025. URL: https://medium.com/@harshithgowdakt/deep-dive-into-tcp-sockets-how-data-travels-under-the-hood-7c16f6b2bf95 (besucht am 14. 10. 2026).

[5] Heise. *Creating a list of figures*. URL: https://www.heise.de/tipps-tricks/LaTeX-Abbildungsverzeichnis-automatisch-erstellen-7261837.html (besucht am 13. 10. 2026).

[6] Freddy Mcloughlan. *Implementing biber for biblatex in Microsoft Visual Studio Code*. 2024. URL: https://tex.stackexchange.com/questions/459640/implementing-biber-for-biblatex-in-microsoft-visual-studio-code (besucht am 13. 10. 2026).

[7] Overleaf. *Creating a bibliography*. URL: https://docs.overleaf.com/citing-and-references/creating-a-bibliography (besucht am 13. 10. 2026).

[8] Overleaf. *How do I adjust the font size?* URL: https://www.overleaf.com/learn/latex/Questions/How_do_I_adjust_the_font_size%3F (besucht am 13. 10. 2026).

[9] Overleaf. *Learn LaTeX in 30 minutes*. URL: https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes (besucht am 12. 10. 2026).

[10] Overleaf. *Page size and margins*. URL: https://www.overleaf.com/learn/latex/Page_size_and_margins (besucht am 13. 10. 2026).

[11] ron rothman. *Converting epoch time into the datetime*. 2023. URL: https://stackoverflow.com/questions/12400256/converting-epoch-time-into-the-datetime (besucht am 09. 10. 2026).

[12] TKDocs. *Tkinter Tutorial*. URL: https://tkdocs.com/tutorial/ (besucht am 23. 09. 2026).

[13] W3Schools. *Python Global Variables*. URL: https://www.w3schools.com/python/python_variables_global.asp (besucht am 07. 10. 2026).

[14] W3Schools. *Python Tuples*. URL: `https://www.w3schools.com/python/python_tuples.asp` (besucht am 04. 10. 2026).